

UNIT 2: WEB DATA EXTRACTION AND API INTEGRATION USING LCNC PLATFORMS

- Web scraping (web harvesting, web data extraction) is the automated process of extracting data from websites using programs or tools instead of manual copy-paste.
- It allows you to collect large volumes of web data and store it in a structured format for further analysis or integration into other systems.
- Web scraping refers to writing scripts or using tools that send HTTP requests to web servers, download HTML pages, parse those pages, and extract specific pieces of information such as titles, prices, or links.
- The main goal is to convert unstructured or semi-structured web content (HTML) into structured data (tables, CSV, JSON, databases).

- Example:

Suppose you want the latest smartphone prices from an e-commerce site.

- Instead of manually browsing and recording each price, a scraper can automatically fetch the product names, prices, and ratings and save them into a CSV file.

Why is Web Scraping Important?

Data-driven decision making:

- Many businesses rely on external data such as competitors' prices, customer reviews, and news headlines.

Automation:

- Scrapers automate repetitive data collection tasks, saving time and reducing human errors.

Integration:

- Scraped data can be fed into dashboards, machine learning models, or internal tools to generate insights.

What is Web Scrapping Used for?

Web Scrapping has multiple applications across various industries.

- 1. Price Monitoring
 - <https://pricehistoryapp.com>
- 2. Market Research
 - <https://socialblade.com>
- 3. News Monitoring
- 4. Sentiment Analysis (Instagram post suggestion algorithm)
- 5. Email Marketing (Scam mails and Calls)

 SEMRUSH

Search

Start free trial

Pricing

Enterprise

More

S

 Home

SEO

Dashboard

Site Performance

Site Audit

Position Tracking

Competitive Analysis

Domain Overview

Organic Rankings

Top Pages

Compare Domains

Keyword Gap

Backlink Gap

Keyword Research

Keyword Overview

Keyword Magic Tool

Keyword Strategy Builder

Content Ideas

SEO Writing Assistant

Topic Research

SEO Content Template

Home > SEO

Send feedback

Create SEO Project

Share

Settings

SEO Dashboard: **Blacksheep.com**

AI Search

United States

AI Visibility

Mentions

Cited pages

ChatGPT

AI Overview

AI Mode

Gemini

Authority Score

Organic Traffic

Organic Keywords

Semrush Rank

Ref. Domains

Backlinks

Position Tracking

United States (Google) • English

Gathering keywords data

0/1

Data collection will take up to 5 minutes. We'll send you an email once it's finished.

Our proprietary ranking of domains with the highest organic visibility. It compares their monthly organic search traffic in a specified region. This is a **Semrush Rank** report.

17°C Sunny

Search

ENG IN

09:35 AM 29-01-2026

Common Use Cases

- Price monitoring and comparison for e-commerce.
- Collecting job listings to analyze demand for particular skills.
- Gathering news or blog content for sentiment analysis.
- Monitoring social media or forums for brand mentions

Web Scraping vs Web Crawling

Web scraping:

- Focuses on extracting specific data from select pages or sites.

Web crawling:

- Systematically discovers and indexes many pages across sites, often as a first step in building search engines (e.g., search engine crawlers).

Type Of WebScraper

Based on Development Type

Self-built Web Scrapers

- Created from scratch using programming languages like Python or JavaScript.
- Require advanced coding knowledge.
- Offer full customization and flexibility.
- More features demand deeper technical expertise.

• Pre-built Web Scrapers

- Already developed tools that can be easily downloaded and run.
- Offer user-friendly interfaces and advanced customization options.
- Suitable for users with little or no coding experience.

Based on Platform

- **Browser Extension Web Scrapers**

- Installed directly as extensions in browsers like Chrome or Firefox.
- Easy to use and quick to set up.
- Limited by browser capabilities — cannot perform complex or large-scale scraping tasks.

- **Software Web Scrapers**

- Standalone applications installed on your computer.
- More advanced and feature-rich than browser-based scrapers.
- Not limited by browser restrictions but require installation and system resources.

Based on Execution Environment

- **Cloud Web Scrapers**

- Operate on cloud servers provided by scraper vendors.
- Don't use your computer's CPU or RAM.
- Allow multitasking since data scraping runs remotely.

- **Local Web Scrapers**

- Run directly on your own computer.
- Depend on local system resources (CPU, RAM).
- May slow down your system during heavy scraping tasks.

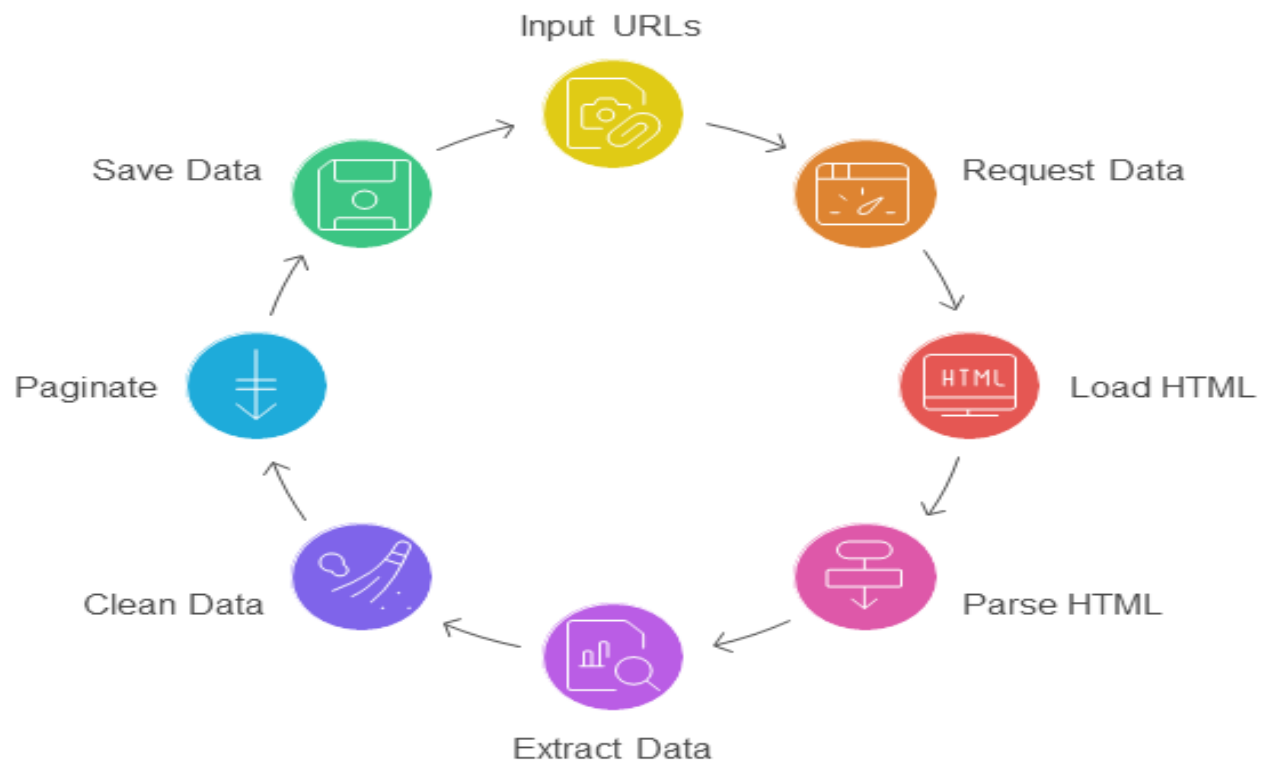
Legal and Ethical Aspects

- Always respect website terms of service and robots.txt directives, which specify which paths are allowed or disallowed for automated access.
- Avoid scraping personal or sensitive data and comply with privacy regulations such as GDPR where applicable.
- Use polite scraping practices like rate limiting and identifying your bot via user-agent to avoid overloading servers.

Web Scrapping End-to-End Flow

- **Input:** Give URLs + specify what data you want (e.g., product name & price only).
- **Request:** Scraper visits each URL like a browser (sends HTTP GET).
- **Load:** Downloads HTML (runs JavaScript if needed for dynamic pages).
- **Parse:** Turns HTML into a navigable structure.
- **Extract:** Finds & pulls only the targeted data using selectors/XPath.
- **Clean:** Trims, converts, and organizes data into rows.
- **Paginate:** Follows “Next” links and repeats until done.
- **Save:** Exports clean data as CSV, Excel, JSON, or database.

Web Scraping Cycle



HTML DOM Basics

- To extract data from web pages, you must understand HTML structure and the Document Object Model (DOM).
- Scrapers typically parse the HTML DOM and navigate it to find the elements of interest.

What is HTML?

- HTML (HyperText Markup Language) is the standard language used to create web pages.
- It uses tags like `<html>`, `<head>`, `<body>`, `<div>`, `<a>`, `<p>`, `<table>` to structure content.

Example snippet:

```
<html>  
  <head><title>My Page</title></head>  
  <body>  
    <h1 class="title">Product List</h1>  
    <div class="product">  
      <span class="name">Phone X</span>  
      <span class="price">₹30,000</span>  
    </div>  
  </body>  
</html>
```

What is DOM?

- DOM (Document Object Model) is a tree-like representation of the HTML document in memory, where each tag becomes a node with parent–child relationships.
- Browsers and scraping libraries use the DOM to traverse and manipulate elements.

Key points:

- Root node: `<html>`
- Child nodes: `<head>`, `<body>` etc.
- Every element can have attributes such as `id`, `class`, `href`, `src`.

Understanding DOM is crucial because scrapers use selectors (such as CSS selectors or XPath) to locate nodes.

Important HTML Elements for Scraping

- Text containers: `<p>`, `<span>`, `<div>`.
- Headings: `<h1>` to `<h6>`.
- Links: `<a href="...">`.
- Images: ``.
- Lists: `<ul>`, `<li>`, `<ol>`.
- Tables: `<table>`, `<tr>`, `<td>`.

Scrapers often target elements by tag, class, id, or attribute.

CSS Selectors and Attributes

Most scraping libraries support CSS selectors to locate elements:

- Select by tag: `h1`, `p`.
- Select by class: `.title` for `class="title"`.
- Select by id: `#main` for `id="main"`.
- Nested selection: `.product .price` = elements of class `price` inside elements of class `product`.

Example: To extract the price from the earlier HTML, a Python script using BeautifulSoup might use `soup.select_one('.product .price').get_text()`.

3. Static vs Dynamic Scraping

- Web pages are either static (simple HTML) or dynamic (content loaded by JavaScript).
- This difference strongly affects how you design scrapers.

Static Web Pages

- In a static page, server sends fully rendered HTML to the client.
- The content you see in “View Page Source” is the same content you need to extract.
- Scraping static pages is straightforward using libraries that fetch HTML such as Requests + BeautifulSoup in Python.

Typical steps:

1. Send HTTP GET request to page URL.
2. Receive HTML response.
3. Parse HTML using parser library.
4. Extract required elements using CSS selectors or XPath.

Dynamic Web Pages

- In dynamic pages, the initial HTML may be minimal, and additional content (e.g., product lists, comments) loads via JavaScript after page load using techniques like AJAX and fetch API.
- “View Page Source” may not show all data; instead, data is fetched from API endpoints in the background.

To scrape dynamic content, typical approaches:

- Use a browser automation tool like Selenium, Playwright, or Puppeteer that executes JavaScript and allows you to access the fully rendered DOM.
- Directly call the underlying API endpoints that the page calls, if they are accessible and permitted by terms of service.

Static vs Dynamic Scraping

Aspect	Static Scraping	Dynamic Scraping
Page type	Plain HTML, minimal JS	Heavy JavaScript, SPA (Single Page Apps)
Data location	In server-generated HTML	Loaded via JS from APIs
Tools	Requests, BeautifulSoup, Cheerio, etc.	Selenium, Playwright, Puppeteer, or API calls.
Complexity	Lower	Higher
Performance	Generally faster	Slower (full browser or multiple API calls)
Common examples	Blog pages, documentation	Infinite scrolling feeds, dashboards, search results

Data Storage (CSV, Google Sheets)

- After extracting data, you must store it in some form for analysis or sharing. Common choices are CSV files and Google Sheets.

CSV (Comma-Separated Values)

- CSV is a simple text format where each line represents a row and each value is separated by a comma (or another delimiter like semicolon).
- It is widely supported by Excel, LibreOffice, databases, and data analysis tools.

Advantages:

- Easy to read and write programmatically.
- Lightweight, human-readable, and tool-independent.
- Ideal for batch exports from scrapers.

Example CSV:

name,price,rating

Phone X,30000,4.5

Phone Y,25000,4.2

- Most web-scraping tutorials demonstrate saving scraped data directly to CSV via file operations in Python or JavaScript.

Google Sheets

- Google Sheets is a cloud-based spreadsheet application that can act as an online database or reporting layer.
- You can push scraped data into Google Sheets via APIs, integration tools (like Parabola), or extensions.

Benefits:

- Easy sharing and collaboration.
- Real-time editing and version history.
- Integration with Google Data Studio, Google Apps Script, etc.

For LCNC context, many no-code and low-code tools let you connect scrapers or APIs directly to Google Sheets without writing code.

Other Storage Options

- Relational databases (MySQL, PostgreSQL).
- NoSQL stores (MongoDB).
- Data warehouses (BigQuery, Snowflake).

Introduction to Web Scraping Tools and Libraries

Traditional scraping uses programming languages and libraries, but LCNC emphasises tools that reduce or remove code.

1. Popular Coding-Based Libraries

Even in LCNC context, you should know names and basic roles:

- Requests: Sends HTTP requests and receives responses (HTML).
- BeautifulSoup: Parses HTML and allows easy navigation and extraction of elements.
- Selenium / Playwright / Puppeteer: Browser automation for dynamic sites.
- Node.js Axios + Cheerio: JavaScript stack for scraping; Axios for requests, Cheerio for DOM parsing.

These libraries implement the standard pipeline: fetch HTML, parse DOM, select elements, and extract data.

2. No-Code / Low-Code Scraping Tools

Several platforms allow scraping via graphical interfaces:

- Tools that provide visual “point-and-click” selection (you click a page element, and the tool learns what to extract).
- Built-in pagination and scheduling.
- Export options to CSV, Google Sheets, or APIs.

LCNC platforms typically integrate scraping features with further data transformation and workflow automation.

3. Best Practices for Tool Selection

- Consider whether the target site is static or dynamic.
- Check if the tool supports login, cookies, and sessions if needed.
- Ensure compliance with robots.txt and legal terms.
- Evaluate export options (CSV, Sheets, databases).

6. Web Scraping – Simple Scraper

A “simple scraper” usually refers to a basic program or flow that targets a small, static site and extracts a limited set of fields.

1. Basic Scraping Pipeline

Nearly every simple scraper follows this pipeline:

1. Identify target site and data.
2. Inspect HTML structure using browser developer tools.
3. Send HTTP request to fetch the page HTML.
4. Parse HTML into a DOM representation.
5. Locate elements (e.g., by CSS selector).
6. Extract and clean text or attribute values.
7. Store data into CSV or Google Sheets.

2. Example: Simple Product Scraper

Assume we are scraping a static list of products from a site.

- Target data: product name, price, rating.
- Inspect HTML: find that products use `<div class="product">`, name in `<span class="name">`, price in `<span class="price">`.
- Code (conceptual, not full):
 - Use Requests to `GET` the URL.
 - Use BeautifulSoup to parse HTML.
 - Loop over `div.product` elements and extract text from `.name` and `.price`.
 - Write rows into a CSV.

3. Error Handling and Robustness

Even a simple scraper should consider:

- HTTP errors (status codes other than 200).
- Changes in page structure (missing elements).
- Slow or blocked requests (timeouts, captchas).

4. Ethics in Simple Scraper Design

- Start with small scale (few pages) to test before scaling up.
- Respect rate limits and robots.txt.
- Provide attribution where appropriate.

7. Scraper Setup

“Scraper setup” means preparing your environment, tools, and configurations before writing or running the scraper.

7.1 Environment Setup

Install required language (e.g., Python) and libraries such as Requests and BeautifulSoup.

Confirm that you can run simple scripts from the command line or IDE.

- Use virtual environments to manage dependencies if working in a larger project.

command-line commands like `pip install requests beautifulsoup4` for Python or `npm install axios cheerio` for Node.js.

7.2 Understanding the Target Site

Before writing code:

- Read the site's robots.txt to check allowed paths.
- Manually browse and use browser developer tools (Elements, Network tab) to understand where data comes from.
- Identify whether data is directly in HTML or loaded via API calls.

7.3 Configuration Parameters

- Base URL(s) to scrape.
- Pagination pattern (e.g., ?page=1, ?page=2).
- Selectors for key elements (`.product .name`, `.product .price`).
- Output location (file path for CSV, sheet name for Google Sheets).

In a no-code tool, these settings are usually entered through forms or drag-and-drop components rather than code.

7.4 Scheduling and Automation

Once setup is done, you may schedule scrapers:

- Run daily or hourly to track changes (e.g., prices).
- Use cron jobs or platform schedulers to execute flows automatically.

Automation platforms like Parabola allow defining schedules or triggers for workflows that include scraping and data transformation.

8. API Fundamentals (REST, JSON)

When dealing with dynamic sites or integrating existing services, you often interact with APIs instead of scraping HTML.

8.1 What is an API?

- API (Application Programming Interface) is a set of rules and endpoints that allow programs to communicate with each other.
- Web APIs accept HTTP requests and return structured data, often in JSON format.
- Example use: A weather API might expose an endpoint `/current?city=Pune` that returns temperature and humidity in JSON.

8.2 REST (Representational State Transfer)

REST is an architectural style commonly used for web APIs.

Key principles:

- Uses stateless client–server communication over HTTP.
- Utilizes standard HTTP methods:
 - GET: retrieve data.
 - POST: create data.
 - PUT/PATCH: update data.
 - DELETE: remove data.
- Resources identified by URLs (e.g., `/users`, `/products/123`).

REST APIs often return JSON responses that are easy to parse and integrate.

8.3 JSON (JavaScript Object Notation)

- JSON is a lightweight data-interchange format based on key–value pairs and arrays.
- It is human-readable and language-independent.

Example JSON:

```
json
{
  "city": "Pune",
  "temperature": 28,
  "units": "Celsius"
}
```

When using APIs for data extraction, you typically send a request and parse the JSON in your code or LCNC tool.

8.4 API vs Web Scraping

- API access: You ask the server for data in a structured format via documented endpoints.
- Web scraping: You fetch HTML pages and extract data from presentation structure.

If a site provides a public or partner API, using it is usually more stable, faster, and legally safer than scraping.

9. Introduction to APIs – Parabola

Parabola is a no-code data automation platform that enables users to connect data sources, transform data, and integrate with APIs without writing custom code.

9.1 What is Parabola?

- Parabola provides an AI-assisted, drag-and-drop workflow builder where each step represents an operation on data (import, transform, export).
- It is designed to automate manual data workflows such as combining spreadsheets, cleaning datasets, and connecting APIs.

Key characteristics:

- No-code / low-code: Built for non-developers or technical users who prefer visual configuration.
- Data-centric: Focus on ETL-style operations (Extract, Transform, Load).

9.2 Parabola Workflow Stages

Typical workflow in Parabola follows three high-level stages:

1. Import / Connect Data Sources

- Connect to CSV files, Google Sheets, databases, or third-party APIs.
- Use built-in connectors (e.g., Shopify, NetSuite) or generic API steps to call REST endpoints.

2. Transform Data

- Apply operations such as filtering rows, merging tables, grouping, aggregations, cleaning text, and parsing nested JSON.
- Use formula or rule steps to define business logic.
- AI-powered steps can assist with parsing unstructured text or transforming formats.

3. Export / Output Data

- Send processed data back to systems like Google Sheets, CRMs, ERPs, or other APIs.
- Automate sending reports via email or pushing metrics to dashboards.

Once a flow is configured, it can be scheduled or triggered automatically to keep data pipelines up to date.

9.3 Using APIs in Parabola

Parabola includes dedicated components for interacting with REST APIs:

- “Pull from API” steps: Configure URL, HTTP method (GET/POST etc.), headers (e.g., authentication tokens), and query parameters. The step will call the API and load JSON data into the flow as a table.
- “Send to API” steps: Use transformed data to create or update records in external systems via POST/PUT/PATCH calls.

9.4 Example Parabola Use Cases

- Pull sales data from Shopify via API, clean and aggregate it in Parabola, and send summarized metrics to Google Sheets.
- Import invoices from PDFs or emails, extract key fields, and call an API to insert them into an ERP system.
- Combine multiple CSV exports, standardize formats, and send the cleaned dataset to a CRM via REST API.

These examples demonstrate how Parabola serves as an LCNC bridge between APIs and business users.

Thank you all.....

Sample Questions

1. What is web scraping and why is it used in data-driven decision making?
2. Differentiate between manual data collection and automated web scraping.
3. List any four real-world applications of web scraping.
4. What is the difference between web scraping and web crawling?
5. Name two tools commonly used for scraping dynamic websites.
6. What is the role of HTML DOM in web scraping?
7. Give examples of important HTML elements targeted during scraping.
8. What are CSS selectors? Provide two examples.
9. Why is CSV commonly used for storing scraped data?
10. Mention two advantages of Google Sheets for storing extracted data.

11. What is an API?
12. Expand REST and explain its purpose.
13. List standard HTTP methods used in REST APIs.
14. What is JSON and why is it widely used in APIs?
15. Explain the end-to-end workflow of web scraping.
16. Compare static scraping and dynamic scraping with suitable examples.
17. Explain the structure of HTML and the concept of the Document Object Model.
18. Discuss different types of web scrapers based on platform and execution environment.
19. Describe the process of building a simple product scraper.
20. Discuss various storage options available after data extraction.
21. Explain the difference between API-based data access and traditional web scraping.
22. How would you extract product name and price from a static e-commerce page?

23. Suggest an approach to scrape data from a website that loads content using JavaScript.
24. How can scraped data be integrated into dashboards or machine learning systems?
25. Design a simple workflow to store scraped data into Google Sheets automatically.
26. How can scheduling and automation improve continuous data monitoring?
27. What is Parabola and how does it support no-code data automation?
28. Explain the three major stages of a Parabola workflow.
29. How does the “Pull from API” step work in Parabola?
30. Describe how transformed data can be sent back to external systems using APIs.
31. Provide one real-world business scenario where Parabola can automate data processing.
